NLP GROUP PROJECT · OPTION 1 — APPLICATION DEVELOPMENT

ATC Readback Verifier

Individual Reflections

One reflection per team member, as required by the assignment.

TEAM

Jan · Vlad · Felipe · Alberto · Kishan · Yi

REPOSITORY

github.com/janwejchert/NLP

DATE

June 2026

Individual Reflection — Jan

Role: App / UI & hosting — repo owner (`app/streamlit_app.py` , Streamlit Cloud deploy, CI, Makefile); author of the analysis notebook (`notebooks/analysis.ipynb`).

My specific contributions

I built the demo UI in `app/streamlit_app.py` : a dark control-tower "annunciator" theme where a phosphor-green light reads READBACK VERIFIED on a MATCH and a pulsing red CHECK light fires on a discrepancy. I wrote the two-column instruction/readback inputs, the scenario picker (`EXAMPLES` , e.g. "Transposed runway 21 → 12" and a callsign-error case), the field-by-field readout table mapping each `Discrepancy.category` to a styled status strip, and the "Raw extracted fields (JSON)" expander so reviewers can see exactly what the model returned. I wrapped the extractor in `@st.cache_resource` so it loads once per session. I own the GitHub repo, the `.github/workflows/ci.yml` (ruff + pytest on 3.11/3.12), and the Makefile. I deployed the hosted demo on Streamlit Community Cloud, and I authored `notebooks/build_notebook.py` , which generates `analysis.ipynb` end to end — the unit tests, the comparator on worked examples, the live pipeline, the four-run metrics table, the prompt-iteration experiment, the 3B-vs-7B figures, and the failure analysis.

What I learned (NLP and engineering)

The biggest lesson was that the same code must run in two very different environments. Locally it talks to ollama (`qwen2.5:3b`); on the free Cloud tier — which cannot run a local model — it uses the `hf` backend. I learned that Streamlit Cloud exposes secrets via `st.secrets` , not environment variables, so I wrote `_load_streamlit_secrets()` to bridge `EXTRACTOR_BACKEND` and `HF_TOKEN` into `os.environ` , letting `get_extractor` stay backend-agnostic. On the NLP side, building the notebook taught me to read evaluation as evidence: extraction is the bottleneck, not Vlad's comparator, and the regressed "hardened" run literally doubled the false-alarm rate.

Challenges and how I handled them

Rendering the field table cleanly was fiddly: the schema's normalization lower-cases callsigns, which looked wrong on screen, so `_display()` falls back to `fields.raw["callsign"]` for the original casing while comparison still uses the normalized form. I also escaped every model-derived string with `html.escape` since I inject raw HTML for the theme. Making the notebook reproducible meant loading committed `eval/results/runs/` artifacts and guarding the live-Ollama cell so it runs without a model.

If I did it again

I'd add a Playwright smoke test in CI that loads an example and asserts the verdict, and surface model latency in the status bar. I'd also cache HF responses to soften free-tier rate limits during the demo.

Use of AI tools (personal note)

I used an AI assistant to draft CSS for the annunciator theme and to explain Streamlit's secrets model, but I verified every claim by deploying to Cloud and watching the `hf` backend actually run. I confirmed the notebook regenerates by running `python notebooks/build_notebook.py` then `nbconvert --execute` and checking the numbers matched our committed metrics.

Individual Reflection — Vlad

Role: Comparator core IP — the deterministic field-by-field comparator, the typed 8-field schema and normalization, the 5-category taxonomy, and the comparator unit tests (`src/atc_verifier/compare.py` , `verdict.py` , `schema.py` , `tests/test_compare.py`).

My specific contributions

I built the deterministic half of the system: the LLM only extracts, and every MATCH-or-discrepancy judgement is my Python. I designed the typed schema in `schema.py` — the 8 fields (`callsign, altitude, heading, speed, frequency, squawk, runway, qnh`), with sub-structures for `Altitude` (FL vs ALT feet), `Heading` , and `Runway` , plus the normalization that makes semantically-equal readbacks compare equal: telephony "one zero one three" → 1013, `normalize_frequency` trimming 118.70 → 118.7, `normalize_squawk` preserving leading zeros via `zfill(4)` , and the policy that wind is informational and must never map to heading/speed. In `compare.py` I wrote `compare_fields` and the per-field rules, and I defined the 5-category taxonomy (`value_substitution, digit_transposition, omission, callsign_error, added_element`). I wrote the 26 unit tests in `tests/test_compare.py` , which run with no LLM.

What I learned (NLP and engineering)

The biggest lesson: extraction is the bottleneck, not comparison. Across the whole iteration the comparator was *never* the cause of a failure — the one residual miss (TC12) was an extraction miss the model handed me, not a logic error. I also learned to separate failure *modes* that look similar: `_is_transposition` flags a digit reorder only when two values share the same digit multiset in a different order (squawk 5701 → 5071 is a transposition; FL240 → FL250 is a substitution), which matters because a transposition is a distinct, higher-risk readback error worth its own category.

Challenges and how I handled them

The runway false alarms. Small models hallucinate an L/R/C side, so comparing a stated side against an invented one fired spurious discrepancies. My fix in `_compare_runway` was principled: compare the runway *number* first, and flag a side difference *only when both messages explicitly state a side*. That single rule killed the runway-side false alarms and was the comparator-side half of the final fix that brought the false-alarm rate down to 6.2%. Callsign I kept strict — `_compare_callsign` treats any mismatch as a `callsign_error` because it is the aircraft's identity, even if the digits happen to be a transposition.

If I did it again

I'd add a confidence/abstain signal so that when extraction is shaky (the TC12 situation) the comparator can surface "uncertain" rather than silently trusting a partial field, and I'd grow `tests/test_compare.py` with property-based tests over normalization edge cases.

Use of AI tools (personal note)

I used an AI assistant to draft docstrings and brainstorm edge cases for transposition detection, but I verified every rule myself by writing the 26 unit tests in `tests/test_compare.py` and stepping through real cases like TC07, TC12, and TC33 by hand against the gold labels. Where the assistant suggested over-clever logic, I rejected it in favour of code I could defend line-by-line in the Q&A.

Individual Reflection — Felipe

Role: Extraction & prompts — owner of `src/atc_verifier/extract/` (the `Extractor` interface and factory, both backends, and the few-shot prompt `prompts/extract_fields.txt`).

My specific contributions

I built the extraction layer that turns one radio message into our 8-field schema. In `base.py` I wrote the abstract `Extractor` interface and the `get_extractor` factory, which reads `EXTRACTOR_BACKEND` and lazily imports either `OllamaExtractor` (local qwen2.5:3b, temperature 0 for determinism) or `HuggingFaceExtractor` (Qwen2.5-7B-Instruct, for Jan's hosted demo) so the app never needs both

dependency sets. I wrote the shared `parse_model_json` helper — it strips code fences, grabs the first balanced `{...}` block, and returns `{}` rather than crashing — plus the one stricter "Return ONLY the JSON object" retry in `extract()`. Centralising cleanup here means both backends behave identically. I also authored and iterated the few-shot prompt itself.

What I learned (NLP and engineering)

The big lesson was that prompt engineering for a small model is not "add more examples." During a hardening pass I added more few-shot cases and the false-alarm rate doubled to 25% — the 3b model started reading the `Easy 4471` callsign suffix as a squawk and `runway 24` as `heading 240`. The extra examples poisoned it. The fix that stuck was safe *rules*, not examples: squawk comes only from an explicit `squawk` instruction (never callsign digits), never guess a runway side, and wind ("wind 250 degrees 8 knots") is informational, never a heading or speed. I learned that with small models, constraints generalise where examples overfit.

Challenges and how I handled them

Extraction is the bottleneck — every residual failure was a missed field, never a comparator bug. TC12 still fails on 3b as a pure extraction miss, and 7B reads it correctly. I leaned on temperature 0 and `parse_model_json` so non-determinism and malformed JSON never contaminated Alberto's eval runs.

If I did it again

I'd add a tiny per-field regression harness for extraction alone (e.g. asserting `Easy 4471` keeps `squawk: null`), so prompt edits get caught before they reach the comparator. I'd also try a constrained-decoding / JSON-schema mode to retire the brace-matching fallback.

Use of AI tools (personal note)

I used an AI assistant to draft candidate prompt rules and brainstorm failure hypotheses for the poisoning regression, but I verified every change by re-running extraction on the offending cases (TC07, TC12, TC33) and reading the raw JSON myself before trusting any false-alarm number.

Individual Reflection — Alberto

Role: Evaluation & metrics — owner of the evaluation harness (`eval/run_eval.py`), the metric definitions (`eval/metrics.py`), and the reproducible runs in `eval/results/` .

My specific contributions

I built the harness that turns our 50-case labelled set into numbers. The central design decision was framing the **positive class as "the readback contains an error"** (see the module docstring and `compute_metrics` in `eval/metrics.py`), so precision, recall and F1 measure *error detection*, not generic accuracy. On top of the standard 2×2 confusion matrix I added the metric I care about most: the **false-alarm rate** in `compute_metrics` — `fp / (fp + tn)` , the fraction of genuinely-correct readbacks we wrongly flag. I also added **per-category detection recall** so we can prove we catch every one of the five taxonomy categories (value_substitution, digit_transposition, omission, callsign_error, added_element), and the `EvalRecord` properties `category_correct` / `fields_correct` for lenient category and affected-field scoring. In `run_eval.py` I made runs reproducible (temperature 0, pinned model tag, a `--out` flag so each run archives under `eval/results/runs/`) and wrote `predictions.csv` carrying the model's raw extracted JSON for both messages, plus an auto-generated `failures.md` . I produced the baseline, hardened, final and 7b runs that quantified the iteration: a +12.5pp false-alarm regression on hardening (12.5% → 25.0%), then a **−18.8pp recovery (25.0% → 6.2%)** from the principled final fix.

What I learned (NLP and engineering)

The biggest lesson was making "the prompt feels better" *measurable*. Our "hardening" attempt subjectively read like an improvement, but my harness showed it regressed false-alarm from 12.5% to 25%. Without a committed metric and the raw-JSON audit trail, that regression ships silently. I also learned why in a safety setting false-alarm rate dominates raw accuracy: a verifier that cries wolf gets switched off, so a 6.2% false-alarm rate is a *headline*, not a footnote.

Challenges and how I handled them

Diagnosing failures was hard until I logged each model's extracted fields per case. That turned TC12 from "a wrong verdict" into a visible **extraction miss**, and proved the comparator was never the cause — every residual error lived upstream in extraction. I also had to keep the harness backend-agnostic so the same code scored both ollama (3b) and hf (7b) runs.

If I did it again

I'd add confidence intervals or bootstrap resampling — 50 cases is thin, and one flipped case (TC12) moves F1 noticeably. I'd also script the run-comparison diff instead of eyeballing two `metrics.md` files.

Use of AI tools (personal note)

I used an AI assistant to sketch the per-category aggregation loop and the markdown formatter in `eval/metrics.py`, then verified every formula by hand against the confusion counts (TP 34, FP 1, FN 0, TN 15) and re-derived F1 = 0.986 myself before trusting it. I treated generated code as a draft to audit, never as ground truth.

Individual Reflection — Kishan

Role: Test set & data quality — owner of the 50-case labelled evaluation set (`eval/data/atc_readback_test_set.csv`).

My specific contributions

I built and curated the entire 50-case labelled test set, `atc_readback_test_set.csv` (TC01–TC50), with its eight columns: `id`, `instruction`, `readback`, `expected_verdict`, `error_category`, `affected_field`, `expected_detail`, and `design_note`. I deliberately balanced it at 16 gold MATCH and 34 gold DISCREPANCY so the metrics could not be gamed by a model that simply always cries "discrepancy." I made sure all five error categories were covered with real spread: value_substitution (TC17–TC27), digit_transposition (TC28–TC34), omission (TC35–TC42), callsign_error (TC43–TC47), and added_element (TC48–TC50). I also seeded the hard cases that earned their keep: TC07 (wind is informational, must stay MATCH), TC10 (spelled-out "one zero one three"), TC11 (conditional clearance behind landing traffic), TC33 (runway 21→12, a high-risk transposition reversal), TC41 ("Wilco" — omission of every mandatory item), and TC47 ("12 Quebec" misheard as "12 Golf"). I owned the gold labels themselves and selected the failure cases for the failure-mode analysis.

What I learned (NLP and engineering)

The deepest lesson was that **a label is an argument, not a fact.** Deciding TC07 should be MATCH meant I had to commit, in writing, to which items are *mandatory* readback items versus informational — wind isn't read back, so penalizing its absence would be wrong. That single choice shaped what "correct" even means downstream. I also learned how a small dataset exposes the real bottleneck: TC12, a plain two-item MATCH, is the one residual failure at 3b, and it's an *extraction* miss, never a comparator one — proof that data quality and extraction, not the deterministic logic, set the ceiling.

Challenges and how I handled them

Designing two-error cases like TC27 (altitude *and* heading both wrong) without making the `affected_field` label ambiguous was hard; I used a semicolon convention (`altitude; heading`) so the gold stayed machine-checkable. Adversarial cases like TC33 and TC47 were the riskiest to label, because a sloppy gold would have masked exactly the safety-critical errors we most wanted to catch.

If I did it again

I'd expand beyond 50 cases with more spelled-out-number and accent-style variants, and add near-miss MATCH cases (paraphrased but correct) to stress the false-alarm rate harder than TC10 and TC14 alone do.

Use of AI tools (personal note)

I used an AI assistant to brainstorm candidate phrasings and to sanity-check my category definitions against ICAO readback conventions, but I hand-wrote and hand-verified every gold label myself, cross-reading each instruction/readback pair. Where the assistant proposed a "MATCH," I re-derived the verdict from first principles before trusting it — and that scrutiny is exactly what surfaced borderline cases like TC07.

Individual Reflection — Yi

Role: Report, field review, and project coordination — owner of `docs/report/` and the slide deck.

My specific contributions

I wrote the technical report and authored `docs/report/field_review.md`, the literature review that positions our verifier at the intersection of aviation safety, ATC language understanding, and structured extraction. I built the justification-of-need narrative around primary sources I verified one-by-one: NASA ASRS's "hear-back problem" (Monan, and the 417-report / 29-month study), the FAA/Volpe finding (Cardosi) that controllers catch only ~two-thirds of incorrect read-backs, and ICAO Annex 10 / Doc 4444 / Doc 9432, which define the seven mandatory spoken read-back items (callsign, runway, level/altitude, heading, speed, transponder code, QNH); we added frequency as the eighth typed field, giving our 8-field schema. I anchored our design choice in the ATCO2 callsign/command/value framing and BERT NER baselines, and in HAAWAII's concept-level read-back-error detection (~80–81%), which mirrors our extract-then-compare approach. I articulated the gap — mature ATC speech systems transcribe rather than verify — and coordinated the team, keeping every reported claim tied to Alberto's eval output rather than to the literature.

What I learned (NLP and engineering)

I learned that a field review is an argument, not a list: each citation had to earn its place by justifying a concrete design decision, e.g. ICAO mandatory items defining our field set, and HAAWAII's finding that semantic-concept comparison is more robust than word-level matching justifying why Vlad's deterministic comparator beats string-equality. On the NLP side, the ATCO2 baselines taught me why command extraction is the hard sub-task and why our 8 typed fields are a tractable scope. I also learned to separate fallible extraction from auditable judgement when explaining the system to non-specialists.

Challenges and how I handled them

The hardest part was citation integrity. Several attractive figures (e.g. specific TRACON percentages) lived only in secondary summaries, so I flagged them in-text as "confirm against the primary PDF" rather than assert them. I also had to keep the report honest about our one residual failure, TC12 — an extraction miss, not a comparator error — and frame the iteration story (FA 12.5% to a regressed 25% to

a principled 6.2%) as evidence that small models can be poisoned by over-specific few-shot examples, not as a comparator weakness.

If I did it again

I would lock the field set against ICAO sources before any prompt work began, and I would version every claim in the report next to the exact eval run that produced it, so a reviewer in Q&A can trace 0.986 F1 straight to the harness output.

Use of AI tools (personal note)

I used an AI assistant to draft prose and suggest related literature, but I treated every reference as unverified until I opened the original — checking the 417-report figure in the NASA TRS PDF and the two-thirds catch rate in the Cardosi study myself. Suggested sources I could not confirm against a primary document were cut, not softened.